

Esercizio S2-L4: Python & AI per la Cyber Security

Obbiettivo:

Imparare i fondamenti dell'interazione con gli LLM tramite Python:

1. Configurare l'ambiente Python + Gemini API (virtual environment, google-genai, pandas).
2. Creare uno script "Hello Gemini" che invia un prompt e stampa la risposta.
3. Creare un Chatbot Security Assistant interattivo con system prompt e ciclo di conversazione.

```
main.py  lab2.py  lab3.py
main.py > ...
1
2
3 from google import genai
4 import os
5
6 # Configura client con API Key da variabile d'ambiente
7 client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
8
9 from google.genai import types
10
11 safety_settings = [
12     types.SafetySetting(category="HARM_CATEGORY_DANGEROUS_CONTENT", threshold="BLOCK_NONE"),
13     types.SafetySetting(category="HARM_CATEGORY_HATE_SPEECH", threshold="BLOCK_NONE"),
14     types.SafetySetting(category="HARM_CATEGORY_HARASSMENT", threshold="BLOCK_NONE"),
15     types.SafetySetting(category="HARM_CATEGORY_SEXUALLY_EXPLICIT", threshold="BLOCK_NONE"),
16 ]
17
18 config = types.GenerateContentConfig(
19     temperature=0.3,          # 0=deterministico, 1=creativo
20     max_output_tokens=1024,    # Lunghezza massima risposta
21     system_instruction="""
22     Sei un assistente amichevole per studenti di programmazione.
23     Rispondi in modo semplice e chiaro.
24     Usa esempi pratici quando possibile.
25     """,
26     safety_settings=safety_settings
27 )
28
29 # Loop per conversazione continua
30 while True:
31     domanda = input("Fammi la tua domanda: ")
32
33     if domanda.lower() == "esci":
34         print("Arrivederci!")
35         break
36
37     response = client.models.generate_content(
38         model="gemini-flash-latest",
39         contents=domanda,
40         config=config
41     )
42     # Stampa output
43     print(f"AI: {response.text}\n")
```

Descrizione Operato:

1. **Importazione delle Librerie:** Per prima cosa da **google** ho importato **genai** (la libreria di Google per usare i modelli Gemini su Python), **os** (per leggere le variabili d'ambiente del sistema) e **types** (per configurare parametri avanzati come la sicurezza e la generazione).

```
from google import genai
import os
from google.genai import types
```

2. **Configurazione del Client:** Ho quindi creato il client che si connette alle API di Google Gemini, inserendo la mia Gemini API Key dalla variabile d'ambiente precedentemente installata tramite il comando **export GEMINI_API_KEY="la mia API KEY"** di modo da non dover andare a inserire l'intera chiave all'interno del

```
# Configura client con API Key da variabile d'ambiente
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
```

codice.

3. **Impostazioni di Sicurezza:** Ho quindi disabilitato tutti i filtri di sicurezza di Gemini (contenuti pericolosi, incitamento all'odio, molestie, contenuti sessualmente espliciti) impostando **BLOCK_NONE**. Questo permette al modello di rispondere liberamente senza censure automatiche.

```
safety_settings = [
    types.SafetySetting(category="HARM_CATEGORY_DANGEROUS_CONTENT", threshold="BLOCK_NONE"),
    types.SafetySetting(category="HARM_CATEGORY_HATE_SPEECH", threshold="BLOCK_NONE"),
    types.SafetySetting(category="HARM_CATEGORY_HARASSMENT", threshold="BLOCK_NONE"),
    types.SafetySetting(category="HARM_CATEGORY_SEXUALLY_EXPLICIT", threshold="BLOCK_NONE"),
]
```

4. **Configurazione del Modello:** Ho configurato come si deve comportare il modello, in particolare andando a definire i seguenti parametri:
 - a. **temperature=0.3** → risposte più precise e meno creative
 - b. **max_output_tokens=1024** → lunghezza massima della risposta
 - c. **system_instruction** → ho dato al modello una personalità, definendo il ruolo che andrà ad assumere nelle risposte.

```

config = types.GenerateContentConfig(
    temperature=0.3,          # 0=deterministico, 1=creativo
    max_output_tokens=1024,   # Lunghezza massima risposta
    system_instruction="""
Sei un assistente amichevole per studenti di programmazione.
Rispondi in modo semplice e chiaro.
Usa esempi pratici quando possibile.
    """,
    safety_settings= safety_settings
)

```

5. **Loop di Conversazione:** Ho infine creato un ciclo infinito (`while True`) che:

- a. Chiede all'utente di inserire una domanda;
- b. Se l'utente scrive "esci", termina il programma;
- c. Altrimenti invia la domanda al modello Gemini con la configurazione definita e stampa la risposta.
- d. Una volta stampata la risposta fornisce all'utente la possibilità di fare un'altra domanda, stabilendo così un loop di conversazione continua.

```

# Loop per conversazione continua
while True:
    domanda = input("Fammi la tua domanda o scrivi 'esci' per terminare il programma: ")

    if domanda.lower() == "esci":
        print("Arrivederci!")
        break

    response = client.models.generate_content(
        model="gemini-flash-latest",
        contents=domanda,
        config=config
    )
    # Stampa output
    print(f"AI: {response.text}\n")

```

Per concludere ho verificato il corretto funzionamento del programma, come segue in figura:

```
Fammi la tua domanda o scrivi 'esci' per terminare il programma: esci
Arrivederci!

$ python3 main.py
Fammi la tua domanda: Spiegami brevemente il modello ISO/OSI
AI: Ciao! Immagina il modello ISO/OSI come una sorta di "manuale di istruzioni" universale che permette a due computer di parlare tra loro, anche se
È diviso in 7 livelli (o layer). Ogni livello ha un compito preciso e passa le informazioni a quello successivo.

Ecco una spiegazione semplice, dal livello più alto (quello che vedi tu) a quello più basso (i cavi):

## 1. Applicazione (Layer 7)
È il livello con cui interagisci direttamente. Quando apri il browser o un'app di messaggistica, sei qui.
* Esempio: HTTP (per il web), WhatsApp, Email.

## 2. Presentazione (Layer 6)
Qui i dati vengono "tradotti". Si assicura che il destinatario capisca il formato dei dati (es. compressione o crittografia).
* Esempio: Trasformare i dati in un formato leggibile come JPEG o cifrare una password.

## 3. Sessione (Layer 5)
Gestisce la "telefonata" tra i due computer. Apre, mantiene e chiude la connessione.
* Esempio: Quando ti logghi su un sito e la sessione rimane attiva mentre navighi tra le pagine.

## 4. Trasporto (Layer 4)
Si occupa di come i dati vengono spediti. Decide se dare priorità alla velocità o alla sicurezza che tutto arrivi intero.
* Esempio: TCP (controlla che ogni pezzetto arrivi, come una raccomandata) o UDP (più veloce ma meno preciso, come lo streaming video).

## 5. Rete (Layer 3)
Qui si decide la strada (il percorso) che i dati devono fare per arrivare a destinazione. È il regno degli indirizzi IP.
* Esempio: Il Router di casa tua che indirizza i pacchetti verso internet.

## 6. Collegamento Dati (Layer 2)
Prepara i dati per il viaggio fisico. Controlla gli errori di trasmissione tra due dispositivi vicini.
* Esempio: L'indirizzo MAC della tua scheda Wi-Fi o Ethernet.

## 7. Fisico (Layer 1)
È la parte "tangibile". Qui i dati sono solo impulsi elettrici, segnali radio o impulsi di luce.
* Esempio: Il cavo LAN, la fibra ottica o le onde del Wi-Fi.

---

## Un esempio pratico: Spedire una lettera
Per ricordarlo meglio, pensa a quando spedisce una lettera:
1. Applicazione: Scrivi il messaggio.
2. Presentazione: Lo scrivi in una lingua che il destinatario capisce

Fammi la tua domanda: 
```